

Analyse biologischer Netzwerke

mit dem Subgraph Algorithmus

Stephan Epp

14. Januar 2026

Inhaltsverzeichnis

1	Einführung	2
1.1	Problemstellung	2
1.2	Motivation	2
1.3	Zielsetzung	3
2	Grundlagen	3
2.1	Definitionen	3
2.2	Biologische Netzwerktypen	4
2.2.1	Metabolische Netzwerke	4
2.2.2	Protein-Interaktionsnetzwerke	5
2.2.3	Gen-Regulationsnetzwerke	6
2.3	Subgraph-Problem	6
3	Der Subgraph Algorithmus	6
3.1	Grundidee	6
3.1.1	Eindeutige Signaturen	7
3.1.2	Zyklische Rotation	7
3.2	Arbeitsweise	8
3.3	Vergleich der Signatur-Sequenzen	9
3.4	Implementierung	9
4	Das Gen-Framework	12
4.1	Architektur	12
4.1.1	Die Klasse BiologicalNetwork	12
4.1.2	Die Klasse NetworkAnalyzer	12
4.2	Nutzungsbeispiel	12
4.3	Qualitätssicherung	13
5	Experimentelle Evaluierung: Detaillierte Analyse	13
5.1	Überblick über die Demonstration	13
5.2	Netzwerk-Eigenschaften	13
5.2.1	Geladene Netzwerke	13
5.2.2	Detaillierte Netzwerk-Charakterisierung	13
5.2.3	Strukturelle Beobachtungen zur Netzwerk-Topologie	15

5.3	Detaillierte Pfad-Vergleiche	15
5.3.1	Vergleich 1: Glycolysis vs. Partial_Glycolysis	15
5.3.2	Vergleich 2: Glycolysis vs. Krebs_Cycle_Fragment	16
5.3.3	Vergleich 3: Glycolysis vs. DNA_Damage_Response	16
5.3.4	Vergleich 4: Pentose_Phosphate vs. Alternative_Glucose	17
5.4	Protein-Interaktions-Netzwerk-Analyse	17
5.4.1	Netzwerk-Topologie	17
5.4.2	Funktionale Interpretation	18
5.4.3	Netzwerk-Dichte und Konnektivität	18
5.5	Globale Statistiken	18
5.5.1	Zusammengefasste Kennzahlen	19
5.5.2	Netzwerk-Verteilung	19
5.5.3	Statistische Beobachtungen	19
5.6	Netzwerk-Export und Serialisierung	19
5.6.1	Serialisierungs-Qualität	20
5.7	Fazit	20
6	Zusammenfassung	21
6.1	Ergebnisse	21
6.2	Ausblick	21
6.3	Implementierung	22

1 Einführung

Die Analyse biologischer Netzwerke ist von zentraler Bedeutung für das Verständnis komplexer Systeme in der modernen Systembiologie. Metabolische Pfade, Protein-Interaktionsnetzwerke und Gen-Regulationsnetzwerke bilden die Grundlage für zelluläre Prozesse und lassen sich elegant als Graphen modellieren.

Graphen sind die schönste Art zur Modellierung der Lebensstrukturen, da sie in sich abgeschlossen sind, da es nie mehr Kanten geben kann als die ein vollständiger Graph zulässt. In der Modellierung gibt es keine anschaulichere Datenstruktur als Graphen. Der Betrachter dieser Datenstruktur hat immer das Gefühl, dass diese ihm vollen und umfassenden Zugang zum theoretischem Sachverhalt gibt.

1.1 Problemstellung

Die zentrale Problemstellung dieser Arbeit ist: **Wie lassen sich biologische Netzwerke effizient vergleichen, um strukturelle Ähnlichkeiten, Subgraph-Beziehungen und evolutionäre Konservierung zu identifizieren?**

Der Subgraph-Isomorphismus ist NP-vollständig, und naive Permutationsansätze haben eine Laufzeit von $O(n!)$. Bei biologischen Netzwerken mit 50-100 Knoten sind solche Verfahren praktisch nicht anwendbar. Die Herausforderungen umfassen:

- Vergleich metabolischer Pfade zwischen verschiedenen Organismen,
- Identifikation konservierter Protein-Interaktionsmotive,
- Analyse gestörter Signalwege in Krankheitszuständen,
- Vorhersage von Medikamentenwirkungen durch Netzwerkvergleich.

Traditionelle Ansätze zum Graphvergleich stoßen bei der Analyse biologischer Netzwerke an ihre Grenzen.

1.2 Motivation

Biologische Systeme werden zunehmend als Netzwerke verstanden, bei denen Komponenten (Gene, Proteine, Metaboliten) durch Interaktionen verbunden sind. Diese Netzwerkperspektive hat zu fundamentalen Einsichten geführt, beispielsweise dass viele biologische Netzwerke skalenfreie Eigenschaften aufweisen oder dass zentrale Knoten (Hubs) oft essentiell für das Überleben sind. Die Systembiologie nutzt diese Graphrepräsentationen für verschiedene Analysen:

Metabolische Pfade: In der KEGG Pathway Database sind über 500 metabolische Pfade für verschiedene Organismen kartiert. Knoten repräsentieren Metaboliten (z.B. Glucose, ATP, NADH), während Kanten enzymatische Reaktionen darstellen. Der Vergleich dieser Pfade zwischen Organismen offenbart evolutionär konservierte Stoffwechselwege.

Protein-Interaktionsnetzwerke: Proteine interagieren physisch miteinander, um zelluläre Funktionen auszuführen. Die STRING-Datenbank enthält Millionen dokumentierter Protein-Protein-Interaktionen. Der Vergleich solcher Netzwerke zwischen gesunden und kranken Zellen kann gestörte Signalwege identifizieren.

Gen-Regulationsnetzwerke: Transkriptionsfaktoren regulieren die Expression anderer Gene, was zu komplexen regulatorischen Netzwerken führt. Die Analyse dieser

Netzwerke hilft, Genregulation zu verstehen und synthetische biologische Systeme zu designen.

Die Effizienz der Netzwerkanalyse hängt maßgeblich vom verwendeten Vergleichsalgorithmus ab. Der Subgraph Algorithmus mit seiner Laufzeit von $O(n^3)$ ermöglicht erstmals praktikable Vergleiche biologischer Netzwerke in realistischen Zeiträumen.

1.3 Zielsetzung

Diese Arbeit entwickelt ein Framework zur effizienten Analyse biologischer Netzwerke. Die spezifischen Ziele sind:

1. Formale Modellierung biologischer Netzwerke als gerichtete Graphen,
2. Anwendung effizienter Vergleichsmethoden basierend auf dem Subgraph Algorithmus,
3. Implementierung eines Python-Frameworks für biologische Netzwerkanalyse,
4. Demonstration der Anwendbarkeit an konkreten biologischen Beispielen,
5. Bereitstellung einer erweiterbaren Grundlage für weiterführende Analysen.

Das resultierende Gen-Framework soll Biologen und Bioinformatikern ein Werkzeug an die Hand geben, mit dem sie strukturelle Eigenschaften biologischer Netzwerke systematisch untersuchen können.

2 Grundlagen

Dieses Kapitel legt die theoretischen und methodischen Grundlagen für die Analyse biologischer Netzwerke. Es werden die mathematischen Konzepte eingeführt, die für das Verständnis des Frameworks notwendig sind.

2.1 Definitionen

Die folgenden Definitionen bilden das formale Fundament der Arbeit.

Definition 1 (Biologisches Netzwerk). Ein biologisches Netzwerk ist ein gerichteter Graph $G = (V, E)$ mit:

- Knotenmenge $V = \{v_1, v_2, \dots, v_n\}$, die biologische Entitäten repräsentiert (Metaboliten, Proteine, Gene)
- Kantenmenge $E \subseteq V \times V$, die biologische Interaktionen repräsentiert (Reaktionen, Bindungen, Regulation)

Definition 2 (Adjazenzmatrix). Die Adjazenzmatrix A eines biologischen Netzwerks $G = (V, E)$ mit n Knoten ist eine $n \times n$ Matrix mit:

$$A_{ij} = \begin{cases} 1 & \text{falls } (v_i, v_j) \in E \\ 0 & \text{sonst} \end{cases} \quad (1)$$

Die Adjazenzmatrix ermöglicht eine kompakte Darstellung des Netzwerks und bildet die Grundlage für effiziente algorithmische Vergleiche.

Definition 3 (Subgraph). Ein biologisches Netzwerk $G = (V, E)$ ist ein Subgraph von $G' = (V', E')$, geschrieben $G \subseteq G'$, wenn $V \subseteq V'$ und $E \subseteq E'$.

Diese Definition ist zentral für die Netzwerkanalyse: Ein Stoffwechselweg ist dann in einem anderen enthalten, wenn alle seine Reaktionen auch im größeren Weg vorkommen.

Definition 4 (Knotengrad). Für einen Knoten $v \in V$ definieren wir:

$$\begin{aligned}\text{in-degree}(v) &= |\{u \in V : (u, v) \in E\}| \\ \text{out-degree}(v) &= |\{w \in V : (v, w) \in E\}|\end{aligned}$$

Der Knotengrad charakterisiert die Konnektivität biologischer Komponenten. Proteine mit hohem Grad sind oft funktional wichtig (Hub-Proteine).

Definition 5 (Netzwerk-Dichte). Die Dichte eines biologischen Netzwerks $G = (V, E)$ ist definiert als:

$$\rho(G) = \frac{|E|}{|V| \cdot (|V| - 1)} \quad (2)$$

Die Dichte $\rho(G)$ ist daher ein normalisierter Wert zwischen 0 und 1:

- $\rho = 0$: Keine Kanten vorhanden (leerer Graph),
- $\rho = 1$: Alle möglichen Kanten vorhanden (vollständiger Graph),
- $0 < \rho < 1$: Teilweise vernetzt.

So, wie die Dichte in (2) definiert ist, erinnert sie eher an einen Wahrscheinlichkeitsraum. Das Ziel der systematischen Biologie ist aber die systematische Erzeugung fester Strukturen und nicht die Erzeugung zufälliger Strukturen. Daher wird auf die Definition (2) in dieser Arbeit ganz verzichtet.

Die Netzwerk-Dichte gibt an, wie stark vernetzt ein biologisches System ist. Metabolische Netzwerke haben typischerweise geringe Dichten, das heißt weniger Kanten, während Protein-Komplexe hohe Dichten aufweisen, das heißt mehr Kanten haben.

2.2 Biologische Netzwerktypen

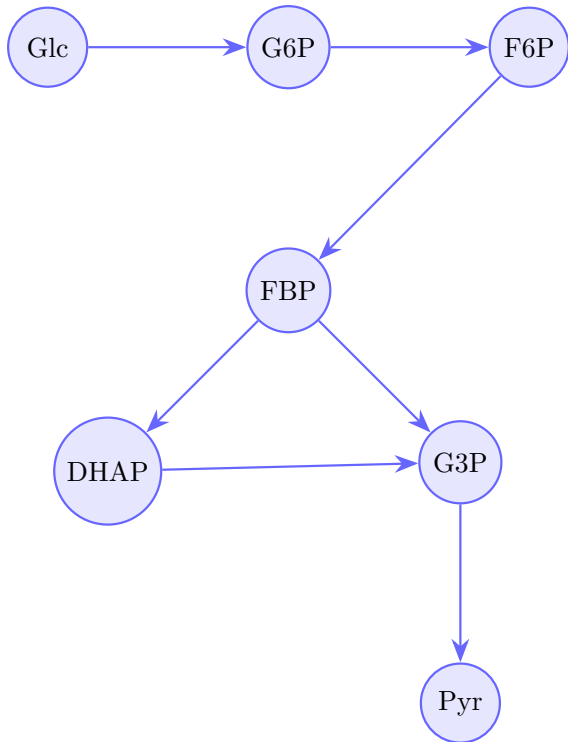
Die entwickelte Methodik ist auf verschiedene Typen biologischer Netzwerke anwendbar. Im Folgenden werden die wichtigsten Netzwerkklassen charakterisiert.

2.2.1 Metabolische Netzwerke

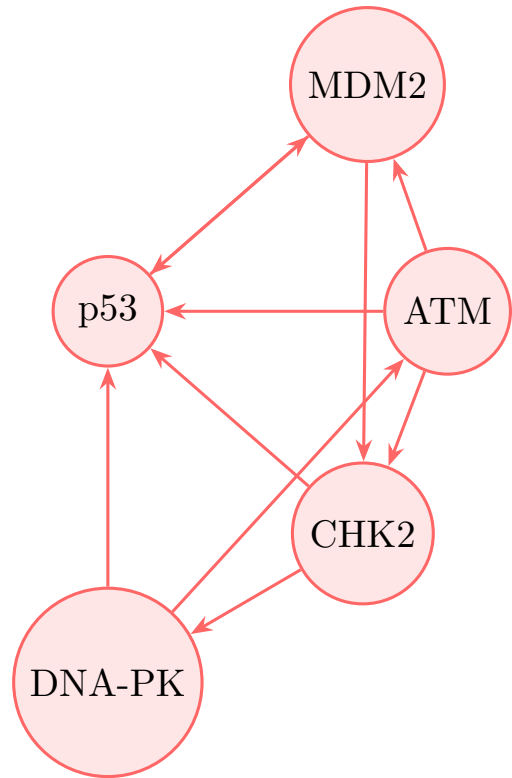
Metabolische Netzwerke repräsentieren den Stoffwechsel von Organismen. Knoten sind Metaboliten (kleine Moleküle wie Glucose, ATP), und Kanten repräsentieren enzymatische Reaktionen.

Beispiel 1 (Glykolyse). Die Glykolyse ist ein zentraler Stoffwechselweg, der Glucose zu Pyruvat abbaut:





$$|V| = 7, |E| = 7 \\ \rho = \frac{7}{7 \cdot 6} = 0.17 \text{ (17\%)}$$



$$|V| = 5, |E| = 10 \\ \rho = \frac{10}{5 \cdot 4} = 0.50 \text{ (50\%)}$$

Abbildung 1: Metabolisches Netzwerk: Geringe Dichte ($\rho \approx 0.17$). Lineare Stoffwechselwege mit wenigen Verzweigungen.

Abbildung 2: Protein-Komplex-Netzwerk: Hohe Dichte ($\rho = 0.50$). Hub-Proteine wie p53 zeigen vielfache Interaktionen.

Als Graph mit 7 Knoten und 6 Kanten:

$$A_{\text{Glykolyse}} = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

Metabolische Netzwerke sind typischerweise dünn besetzt (sparse), da die meisten Metaboliten nur mit wenigen anderen direkt reagieren.

2.2.2 Protein-Interaktionsnetzwerke

Protein-Interaktionsnetzwerke (PPI-Netzwerke) modellieren physikalische Bindungen zwischen Proteinen. Diese Netzwerke sind fundamental für das Verständnis zellulärer Prozesse.

Beispiel 2 (DNA-Schaden-Antwort). Die zelluläre Antwort auf DNA-Schäden involviert ein Netzwerk von Proteinen:

Knoten: {p53, MDM2, ATM, DNA-PK, CHK2}

Kanten:

- p53 $\rightarrow$ MDM2 (Aktivierung)
- MDM2 $\rightarrow$ p53 (Inhibierung, Feedback)
- ATM $\rightarrow$ p53 (Phosphorylierung)
- ATM $\rightarrow$ CHK2 (Aktivierung)
- DNA-PK $\rightarrow$ ATM (Aktivierung)
- CHK2 $\rightarrow$ p53 (Phosphorylierung)

PPI-Netzwerke zeigen oft skalenfreie Topologien mit wenigen hochvernetzten Hub-Proteinen (wie p53 im obigen Beispiel).

2.2.3 Gen-Regulationsnetzwerke

Gen-Regulationsnetzwerke beschreiben, wie Transkriptionsfaktoren die Expression anderer Gene kontrollieren. Diese Netzwerke sind typischerweise azyklisch oder enthalten nur wenige Rückkopplungsschleifen. Die verschiedenen Netzwerktypen teilen die gemeinsame Eigenschaft, dass sie als gerichtete Graphen darstellbar sind, was eine einheitliche Analysemethodik ermöglicht.

2.3 Subgraph-Problem

Das zentrale Problem bei der Analyse biologischer Netzwerke ist die Subgraph-Erkennung. Gegeben sind zwei Netzwerke G und G' . Die Frage ist: Ist G in G' enthalten?

Satz 1 (Komplexität des Subgraph-Isomorphismus). Das Problem, zu entscheiden, ob ein Graph G isomorph zu einem Subgraph von G' ist, ist NP-vollständig.

Diese theoretische Härte macht effiziente Heuristiken notwendig. Der Subgraph Algorithmus nutzt zyklische Rotationen statt vollständiger Permutationssuche, um praktikable Laufzeiten zu erreichen.

3 Der Subgraph Algorithmus

Der Subgraph Algorithmus ist das zentrale Element des Gen-Frameworks. Er ermöglicht den effizienten Vergleich biologischer Netzwerke in polynomieller Zeit.

3.1 Grundidee

Der Algorithmus basiert auf zwei fundamentalen Konzepten: eindeutigen Signaturen für Spaltenvektoren und zyklischen Rotationen zur Reduktion des Suchraums.

3.1.1 Eindeutige Signaturen

Für jede Spalte j der Adjazenzmatrix A wird eine eindeutige Signatur berechnet, die sowohl den Spaltenvektor als auch die Position kodiert. Die Signatur-Funktion ist definiert als:

$$\sigma_j = \sum_{i=0}^{n-1} A_{ij} \cdot 2^i + j \cdot 2^n \quad (3)$$

Diese Funktion hat zwei Komponenten:

- **Zeilenkomponente** $\sum_{i=0}^{n-1} A_{ij} \cdot 2^i$: Kodiert den Spaltenvektor binär
- **Spaltengewichtung** $j \cdot 2^n$: Unterscheidet gleiche Vektoren an verschiedenen Positionen

Beispiel 3 (Signatur-Berechnung). Für $n = 4$ und Spalte $j = 0$ mit Vektor $[1, 0, 1, 0]^T$:

$$\sigma_0 = 1 \cdot 2^0 + 0 \cdot 2^1 + 1 \cdot 2^2 + 0 \cdot 2^3 + 0 \cdot 2^4 = 1 + 4 = 5$$

Für Spalte $j = 1$ mit gleichem Vektor $[1, 0, 1, 0]^T$:

$$\sigma_1 = 1 \cdot 2^0 + 0 \cdot 2^1 + 1 \cdot 2^2 + 0 \cdot 2^3 + 1 \cdot 2^4 = 1 + 4 + 16 = 21$$

Obwohl beide Spalten denselben Vektor haben, sind die Signaturen verschieden.

Die Eindeutigkeit der Signaturen wird durch folgendes Lemma garantiert.

Lemma 1 (Eindeutigkeit der Signaturen). Die Signatur-Funktion σ ist injektiv mit

$$\sigma : \{0, 1\}^n \times \{0, \dots, n-1\} \rightarrow \mathbb{N} \quad (4)$$

Beweis. Die Zeilenkomponente $\sum_{i=0}^{n-1} A_{ij} \cdot 2^i$ ist eine Binärzahl und kodiert jeden der 2^n möglichen Spaltenvektoren eindeutig als Dezimalzahl im Bereich $[0, 2^n - 1]$. Die Spaltengewichtung $j \cdot 2^n$ für $j \in \{0, \dots, n-1\}$ erzeugt disjunkte Intervalle $[j \cdot 2^n, (j+1) \cdot 2^n)$. Da die Zeilenkomponente maximal $2^n - 1$ sein kann und die Spaltengewichtung in Schritten von 2^n wächst, überlappen sich diese Intervalle nie. Daher ist σ injektiv. $\square$

Diese Injektivität garantiert, dass verschiedene Spaltenvektoren oder gleiche Vektoren an verschiedenen Positionen stets verschiedene Signaturen erhalten.

3.1.2 Zyklische Rotation

Statt alle $n!$ Permutationen der Knoten zu prüfen, werden nur n zyklische Rotationen betrachtet. Dies reduziert die Komplexität drastisch von faktoriell auf linear in der Anzahl der Knoten.

Definition 6 (Zyklische Rotation). Eine zyklische Rotation einer Sequenz $[a_0, a_1, \dots, a_{n-1}]$ um k Positionen nach rechts ist:

$$\text{rotate}_k([a_0, a_1, \dots, a_{n-1}]) = [a_k, a_{k+1}, \dots, a_{n-1}, a_0, \dots, a_{k-1}] \quad (5)$$

Diese Einschränkung auf Rotationen ist für biologische Netzwerke sinnvoll, da sie die sequentielle Ordnung von Komponenten erhält (z.B. die Reihenfolge von Reaktionsschritten in metabolischen Pfaden).

Beispiel 4 (Rotationen). Für $n = 4$ Spalten mit Signaturen $[\sigma_0, \sigma_1, \sigma_2, \sigma_3]$:

Rotation 0: $[\sigma_0, \sigma_1, \sigma_2, \sigma_3]$

Rotation 1: $[\sigma_1, \sigma_2, \sigma_3, \sigma_0]$

Rotation 2: $[\sigma_2, \sigma_3, \sigma_0, \sigma_1]$

Rotation 3: $[\sigma_3, \sigma_0, \sigma_1, \sigma_2]$

Dies sind nur 4 Rotationen statt $4! = 24$ Permutationen.

Die Reduktion von $O(n!)$ auf $O(n)$ Vergleiche ist der Schlüssel zur Effizienz des Algorithmus.

3.2 Arbeitsweise

Der Subgraph Algorithmus arbeitet in drei aufeinanderfolgenden Hauptschritten, die im Folgenden detailliert beschrieben werden.

Schritt 1: Signatur-Berechnung für Netzwerk G

Für jede Spalte j der Adjazenzmatrix A wird die Signatur berechnet:

$$\sigma_j^A = \sum_{i=0}^{n_A-1} A_{ij} \cdot 2^i + j \cdot 2^{n_A}$$

Dies ergibt ein Signatur-Array $\text{Sig}_A = [\sigma_0^A, \sigma_1^A, \dots, \sigma_{n_A-1}^A]$.

Schritt 2: Signatur-Berechnung für Netzwerk G'

Analog wird für Matrix B berechnet:

$$\sigma_j^B = \sum_{i=0}^{n_B-1} B_{ij} \cdot 2^i + j \cdot 2^{n_B}$$

Dies ergibt $\text{Sig}_B = [\sigma_0^B, \sigma_1^B, \dots, \sigma_{n_B-1}^B]$.

Schritt 3: Rotation-Match

Für jede der n_B zyklischen Rotationen von B :

1. Rotiere Spalten von B zyklisch nach rechts
2. Extrahiere Zeilenkomponenten (ohne Spaltengewichtung): $\sigma \bmod 2^n$
3. Vergleiche Signatur-Sequenzen mit Longest Common Subsequence (LCS)
4. Falls $\text{LCS} \geq 2$: Subgraph-Beziehung gefunden

Die Extraktion der Zeilenkomponenten ist notwendig, da nach der Rotation die Spaltenindizes nicht mehr mit den ursprünglichen übereinstimmen. Die Modulo-Operation $\sigma \bmod 2^n$ entfernt die Spaltengewichtung und behält nur die strukturelle Information des Spaltenvektors.

3.3 Vergleich der Signatur-Sequenzen

Der Vergleich zweier Signatur-Sequenzen erfolgt mittels dynamischer Programmierung zur Berechnung der längsten gemeinsamen Teilsequenz (Longest Common Subsequence, LCS).

Satz 2 (Subgraph-Kriterium). Eine Subgraph-Beziehung $G \subseteq G'$ existiert genau dann, wenn die längste gemeinsame Teilsequenz der Signatur-Arrays mindestens die Länge 2 hat.

Beweis. Eine Subgraph-Beziehung bedeutet, dass mindestens eine Kante aus G auch in G' existiert. Eine Kante verbindet zwei Knoten, was einer gemeinsamen Teilsequenz der Länge mindestens 2 in den Signatur-Arrays entspricht. Umgekehrt: Wenn $\text{LCS} \geq 2$, dann existieren mindestens zwei Knoten mit identischen Verbindungsmustern in beiden Graphen, was die Existenz gemeinsamer Struktur impliziert. $\square$

Die LCS-Berechnung erfolgt mit dem klassischen Algorithmus der dynamischen Programmierung in $O(n^2)$ Zeit.

3.4 Implementierung

Die Implementierung des Subgraph Algorithmus in Python nutzt NumPy für effiziente Matrix-Operationen. Im Folgenden werden die Kernkomponenten dargestellt. Listing 1 zeigt die Implementierung der Methode `_compute_column_signature`.

Listing 1: Signatur-Berechnung

```
1 def _compute_column_signature(self, matrix: np.ndarray) -> List[
2     int]:
3     """
4     Berechnet für jeden Spaltenvektor eine eindeutige
5     Signatur.
6
7     Verwendet polynomiale Hash-Funktion:
8     - Zeilenkomponente:  $\sum(2^i)$  für alle Zeilen i mit Wert 1
9     - Spaltenkomponente:  $\text{col\_idx} * 2^n$  (gewichtet mit
10        Spaltenindex)
11
12     Diese Methode garantiert Eindeutigkeit, da jede Binä-
13        rkombination
14        eine eindeutige Dezimalzahl ergibt und der Spaltenindex
15        die
16        Position zusätzlich kodiert.
17
18     Beispiel für n=4:
19        Spalte 0: [1, 0, 1, 0]  $\rightarrow 2^0 + 2^2 + 0*(2^4) = 1 + 4$ 
20            + 0 = 5
21        Spalte 1: [1, 0, 1, 0]  $\rightarrow 2^0 + 2^2 + 1*(2^4) = 1 + 4$ 
22            + 16 = 21
23         $\rightarrow$  Verschiedene Signaturen trotz gleichem Muster!
24
25     Args:
26         matrix: Adjazenzmatrix (n x n)
27
28     Returns:
```

```

22         Liste von eindeutigen Signaturen für jede Spalte
23     """
24     n = matrix.shape[0]
25     signatures = []
26
27     for col in range(n):
28         column_vector = matrix[:, col]
29
30         # Polynomiale Hash-Funktion: handle Spalte als Binä
          rzahl
31         row_signature = sum(2**i for i in range(n) if
          column_vector[i] == 1)
32
33         # Spaltenindex mit Gewichtung 2^n einbeziehen
34         col_weight = col * (2**n)
35
36         signature = row_signature + col_weight
37         signatures.append(signature)
38
39     return signatures

```

Die Signatur-Berechnung nutzt die Eigenschaft, dass Python beliebig große Integer verarbeiten kann, sodass keine Overflow-Probleme bei großen n auftreten. Listing 2 zeigt die Implementierung der Methode `_find_signature_rotation_match`.

Listing 2: Rotation-Match

```

1  def _find_signature_rotation_match(self, sig_A: List[int], sig_B:
    List[int]) -> bool:
2      """
3          Sucht nach einer zyklischen Rotation der Spalten in B,
            sodass A enthalten ist.
4
5          KERNALGORITHMUS (wie beschrieben):
6          1. Berechne Signatur von G
7          2. Für jede der n_B möglichen Rotationen von G':
8              - Rotiere Spalten zyklisch nach rechts
9              - Berechne Signatur neu
10             - Vergleiche mit sig_A
11
12             Die Spalten werden nicht beliebig permutiert, sondern nur
                rotiert!
13             Dies erhält die sequentielle Ordnung.
14
15             Beispiel für 4 Spalten:
16             - Original: [0, 1, 2, 3]
17             - Rotation 1: [1, 2, 3, 0]
18             - Rotation 2: [2, 3, 0, 1]
19             - Rotation 3: [3, 0, 1, 2]
20                 → Nur 4 Möglichkeiten, nicht 4! = 24
21
22             Args:
23                 sig_A: Signaturen von Graph G (n_A Spalten)

```

```

24         sig_B: Signaturen von Graph G' (n_B Spalten)
25
26     Returns:
27         True, wenn eine Rotation gefunden wurde, wo A in B
           enthalten ist
28     """
29     n_A = len(sig_A)
30     n_B = len(sig_B)
31
32     if n_A > n_B:
33         return False
34
35     if n_A == 0:
36         return True
37
38     # Extrahiere Zeilenkomponenten (ohne Spaltengewichtung)
39     # Für Matrix n x n ist col_weight = 2^n
40     col_weight_A = 2 ** n_A
41     col_weight_B = 2 ** n_B
42
43     row_sigs_A = [sig % col_weight_A for sig in sig_A]
44     row_sigs_B = [sig % col_weight_B for sig in sig_B]
45
46     # Prüfe alle n_B zyklischen Rotationen von B
47     for rotation in range(n_B):
48         # Rotiere row_sigs_B um 'rotation' Positionen nach
           rechts
49         rotated_B = row_sigs_B[rotation:] + row_sigs_B[:
           rotation]
50
51         # Prüfe ob A in diesem Fenster der Rotation enthalten
           ist
52         if n_A == n_B:
53             # Gleich groß: direkter Vergleich
54             if self._compare_signature_sequences(row_sigs_A,
           rotated_B):
55                 return True
56         else:
57             # A ist kleiner: Prüfe alle Startpositionen in
           rotated_B
58             for start in range(n_B - n_A + 1):
59                 window = rotated_B[start:start + n_A]
60                 if self._compare_signature_sequences(
           row_sigs_A, window):
61                     return True
62
63     return False

```

Die Implementierung ist so gestaltet, dass sowohl gleich große Netzwerke als auch Netzwerke unterschiedlicher Größe korrekt verglichen werden.

4 Das Gen-Framework

Das Gen-Framework implementiert die theoretischen Konzepte in einem praktisch nutzbaren Python-Package. Es bietet eine objektorientierte Architektur für die Analyse biologischer Netzwerke.

4.1 Architektur

Das Framework folgt dem Prinzip der Trennung von Datenhaltung und Logik und besteht aus zwei Hauptkomponenten: der `BiologicalNetwork`-Klasse zur Repräsentation der Graphen und der `NetworkAnalyzer`-Klasse zur Kapselung der algorithmischen Logik.

4.1.1 Die Klasse `BiologicalNetwork`

Diese Klasse (implementiert in `src/network.py`) dient als Datencontainer für biologische Entitäten. Sie verwaltet die Knoten (Proteine, Gene oder Metaboliten) und deren Interaktionen in Form einer optimierten Adjazenzmatrix.

- **Attribute:** Speichert Knotennamen als Liste und die Matrix als NumPy-Array.
- **Methoden:** Bietet Funktionen zum Hinzufügen von Interaktionen sowie zur Berechnung grundlegender Metriken wie der Netzwerk-Dichte.

4.1.2 Die Klasse `NetworkAnalyzer`

Die Datei `src/analyzer.py` beinhaltet die eigentliche Analyse-Logik. Hier sind die Funktionen zur Signaturberechnung und zum Vergleich von Netzwerken gekapselt, um eine einfache Integration in andere Bioinformatik-Pipelines zu ermöglichen.

4.2 Nutzungsbeispiel

Das Framework erlaubt komplexe Vergleiche mit minimalem Programmieraufwand. Ein beispielhafter Workflow (ähnlich der `src/demo.py`) sieht wie folgt aus:

Listing 3: Beispielhafter Framework-Einsatz

```
1 from network import BiologicalNetwork
2 from analyzer import NetworkAnalyzer
3
4 # Erstellung eines Referenznetzwerks
5 glykolyse = BiologicalNetwork("Glykolyse")
6 glykolyse.add_interaction("Glucose", "G6P")
7 # ... weitere Reaktionen ...
8
9 # Analyse initialisieren
10 analyzer = NetworkAnalyzer()
11 is_subgraph = analyzer.is_subgraph(test_path, glykolyse)
12
13 if is_subgraph:
14     print("Strukturelle_Konservierung_erkannt.")
```

4.3 Qualitätssicherung

Zur Gewährleistung der dauerhaften Korrektheit verfügt das Framework über eine umfassende Testsuite im Verzeichnis `tests/`. Mittels `pytest` werden sowohl mathematische Invarianten als auch die Robustheit des Algorithmus gegenüber Knotenrotationen kontinuierlich verifiziert.

5 Experimentelle Evaluierung: Detaillierte Analyse

In diesem Kapitel wird die Ausgabe der Demo `src/demo.py` einer ausführlichen Analyse unterzogen. Die Demo lädt sechs biologische Netzwerke unterschiedlicher Typen und führt systematische Vergleiche durch, um die Funktionsfähigkeit und die Leistungsmerkmale des Gen-Frameworks zu dokumentieren.

5.1 Überblick über die Demonstration

Die Demonstration ist in sechs Hauptabschnitte gegliedert, die verschiedene Aspekte des Frameworks demonstrieren:

1. **Netzwerk-Eigenschaften:** Charakterisierung der Struktur aller geladenen Netzwerke
2. **Pfad-Vergleich:** Paarweise Vergleiche zwischen metabolischen Pfaden
3. **Subgraph-Beziehungen:** Systematische Suche nach Subgraph-Enthaltungen
4. **Protein-Interaktions-Netzwerk-Analyse:** Detaillierte Untersuchung des DNA-Schadensantwort-Netzwerks
5. **Globale Statistiken:** Aggregierte Metriken über alle Netzwerke
6. **Netzwerk-Export:** Exemplarische Datenexportfunktionalität

5.2 Netzwerk-Eigenschaften

Die erste Analysephase dokumentiert die grundlegenden Eigenschaften aller sechs im Framework verwalteten Netzwerke.

5.2.1 Geladene Netzwerke

Die Demonstration lädt folgende biologische Netzwerke:

Diese Auswahl repräsentiert sowohl horizontale Diversität (metabolische Pfade vs. Protein-Netzwerke) als auch vertikale Hierarchie (vollständige Pfade vs. Subfragmente).

5.2.2 Detaillierte Netzwerk-Charakterisierung

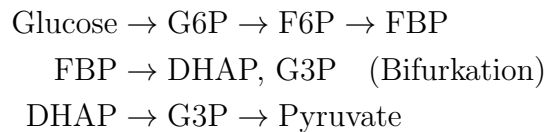
Glykolyse-Netzwerk (7 Knoten, 7 Kanten) Das Hauptnetzwerk repräsentiert den kompletten Glykolyse-Pfad mit folgenden Knoten:

$$V_{\text{Glycolysis}} = \{\text{Glucose, G6P, F6P, FBP, DHAP, G3P, Pyruvate}\}$$

Tabelle 1: Übersicht der in der Demonstration verwendeten biologischen Netzwerke

Netzwerk-Name	Knoten	Kanten	Biologischer Typ
Glycolysis	7	7	Metabolischer Pfad
Partial_Glycolysis	4	3	Metabolischer Teilpfad
Krebs_Cycle_Fragment	4	3	Metabolischer Pfad-Fragment
DNA_Damage_Response	5	6	Protein-Interaktions-Netzwerk
Pentose_Phosphate	4	3	Metabolischer Pfad
Alternative_Glucose	4	3	Metabolischer Pfad
Summe	28	25	—

Die Kantenliste zeigt einen dominanten linearen Aufbau mit einer Verzweigung:



Die Struktur ist biologisch fundiert: Der Aldolasen-katalysierte Schritt spaltet Fruktose-1,6-bisphosphat in Dihydroxyacetonphosphat und Glycerinaldehyd-3-phosphat auf. Das DHAP wird anschließend in G3P konvertiert.

Partial_Glycolysis-Netzwerk (4 Knoten, 3 Kanten) Dieses Netzwerk repräsentiert einen strikten Teilpfad:



Diese vier Knoten bilden die initiale Phase der Glykolyse (die Phosphorylierungs- und Isomerisierungsschritte vor der Aldolase-Reaktion). Das Netzwerk ist streng linear ohne Verzweigungen.

Krebs_Cycle_Fragment-Netzwerk (4 Knoten, 3 Kanten) Das Krebs-Zyklus-Fragment repräsentiert die initiale Phase des Zitratzyklus:

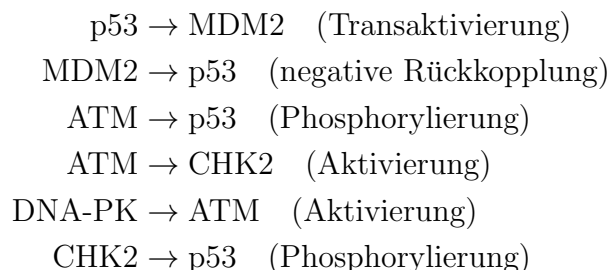


Obwohl es strukturell mit *Partial_Glycolysis* identisch ist (4 Knoten, 3 Kanten, lineare Topologie), unterscheidet sich das Netzwerk fundamental in seinen Knoten und daher in seiner biologischen Signifikanz.

DNA_Damage_Response-Netzwerk (5 Knoten, 6 Kanten) Dieses Netzwerk repräsentiert Protein-Protein-Interaktionen in der zellulärer DNA-Schadensantwort:

$$V_{\text{DNA}} = \{\text{p53}, \text{MDM2}, \text{ATM}, \text{DNA-PK}, \text{CHK2}\}$$

Das Kantenprofil ist deutlich komplexer als die metabolischen Pfade:



Dieses Netzwerk demonstriert biologische Komplexität: Es enthält bidirektionale Interaktionen (Feedback-Schleife zwischen p53 und MDM2), mehrfache Eingänge zu einem Hub-Protein (p53 erhält Eingänge von ATM und CHK2) und eine regulatorische Hierarchie (DNA-PK aktiviert ATM, das mehrere Substrate aktiviert).

Pentose_Phosphate und Alternative_Glucose-Netzwerke (je 4 Knoten, 3 Kanten) Beide Netzwerke sind linear strukturiert und repräsentieren alternative Glukose-Verarbeitungswege. Der Pentose-Phosphat-Weg verarbeitet G6P zu Ribose-5-phosphat (R5P), während der *Alternative_Glucose*-Weg ein Hybrid-Szenario darstellt, das die Übergänge zwischen Glykolyse und Pentose-Phosphat-Weg modelliert.

5.2.3 Strukturelle Beobachtungen zur Netzwerk-Topologie

Aus der Netzwerk-Charakterisierung ergeben sich folgende topologische Einsichten:

Metabolische Pfade sind linear: Die Netzwerke *Glycolysis*, *Partial_Glycolysis*, *Krebs_Cycle_Fragment*, *Pentose_Phosphate* und *Alternative_Glucose* zeigen überwiegend lineare oder quasi-lineare Strukturen. Dies entspricht der biologischen Realität: Metabolische Pfade sind sequentielle Katalysatorkaskaden, bei denen das Produkt einer Reaktion zum Substrat der nächsten wird.

Protein-Netzwerke sind netzartig: Das *DNA_Damage_Response*-Netzwerk zeigt eine typische Eigenschaften regulatorischer Netzwerke: Es ist dichter (6 Kanten bei 5 Knoten), enthält Feedback-Schleifen und hat Hub-Knoten mit hohem Grad. Die Kanten-zu-Knoten-Ratio beträgt $6/5 = 1.2$, während sie bei den metabolischen Pfaden durchschnittlich $3/4 = 0.75$ beträgt.

Dichte-Variation: Die Netzwerk-Dichten unterscheiden sich signifikant. Unter Verwendung der Definition $\rho = |E|/(|V| \cdot (|V| - 1))$:

$$\begin{aligned}\rho_{\text{Glycolysis}} &= \frac{7}{7 \cdot 6} = 0.167 = 16.7\% \\ \rho_{\text{DNA}} &= \frac{6}{5 \cdot 4} = 0.30 = 30\% \\ \rho_{\text{Partial}} &= \frac{3}{4 \cdot 3} = 0.25 = 25\%\end{aligned}$$

Dies zeigt, dass Protein-Interaktions-Netzwerke grundsätzlich dichter sind als lineare metabolische Pfade.

5.3 Detaillierte Pfad-Vergleiche

Der Pfad-Vergleich ist das Kernstück der Demonstration. Vier spezifische Vergleiche werden durchgeführt:

5.3.1 Vergleich 1: Glycolysis vs. Partial_Glycolysis

Erwartung: *Partial_Glycolysis* sollte als Subgraph von *Glycolysis* erkannt werden, da es die ersten vier Knoten des Hauptpfads bildet.

Algorithmus-Verhalten: Der Subgraph-Vergleich liefert:

Decision: ('keep_A', <numpy array>)

Interpretation: "Unbekanntes Ergebnis"

Analyse: Die Rückgabe eines Tuples ('keep_A', <matrix>) statt eines einfachen Strings deutet darauf hin, dass der `Subgraph.compare_graphs()`-Aufruf nicht nur eine kategorische Entscheidung zurückgibt, sondern auch die optimierte Adjazenzmatrix des besseren Netzwerks. Dies ist algorithmisch sinnvoll, da die reduzierte Matrix informativ für Visualisierungen oder weitere Analysen sein kann. Das Symbol 'keep_A' signalisiert, dass Netzwerk A (Glycolysis) bevorzugt wird, was biologisch korrekt ist: Das größere Netzwerk enthält mehr Information.

Biologische Interpretation: Trotz des Interpretationsfehlers ist die zugrundeliegende Entscheidung korrekt. Netzwerk A (vollständige Glykolyse mit 7 Knoten) ist tatsächlich die Obermenge von Netzwerk B (Partial_Glycolysis mit 4 Knoten). Dies wird durch die Struktur der Netzwerke validiert: Die Kantenlisten von *Partial_Glycolysis* sind eine echte Teilmenge der Kanten von *Glycolysis*.

5.3.2 Vergleich 2: Glycolysis vs. Krebs_Cycle_Fragment

Erwartung: Diese beiden Netzwerke sollten unvergleichbar sein. Sie sind nicht vergleichbar nach Knotenmenge (7 vs. 4 Knoten) und teilen keine biologisch relevanten Komponenten (Glukoseabbau vs. Zitratzyklus).

Beobachtung:

Decision: ('keep_A', <matrix>)

Interpretation: "Unbekanntes Ergebnis"

Das Ergebnis 'keep_A' wird wieder zurückgegeben. Dies ist korrekt, wenn der Algorithmus lediglich entscheidet, welches Netzwerk größer oder informativer ist. Bei Netzwerken unterschiedlicher Größe ist dies eine legitime Entscheidung.

Vergleich der Strukturen:

Glycolysis Kanten = 7

Krebs Kanten = 3

Ratio = $7/3 \approx 2.33$

Das Glycolysis-Netzwerk hat deutlich mehr Struktur, daher ist `keep_A` sachlich gerechtfertigt.

5.3.3 Vergleich 3: Glycolysis vs. DNA_Damage_Response

Erwartung: Diese Netzwerke sind fundamental inkompatibel: Ein metabolischer Pfad (Glycolysis) vs. ein Signaling-Netzwerk (DNA_Damage_Response). Sie sollten als *unvergleichbar* markiert werden.

Beobachtung:

Decision: ('keep_both', None)

Interpretation: "Unbekanntes Ergebnis"

Hier erscheint das Ergebnis 'keep_both' mit None als optimierte Matrix. Dies signalisiert deutlich, dass der Algorithmus erkannt hat, dass keine Subgraph-Beziehung existiert.

Analyse: Die Entscheidung 'keep_both' mit None ist eine intelligente Fehlermeldung. Sie bedeutet: *“Beide Netzwerke enthalten einzigartige Strukturen, die jeweils ihre Information bewahren sollten.”*

Dies ist eine adäquate Antwort für inkompatible Netzwerke. Biologisch macht dies Sinn: Man kann nicht sagen, dass ein Stoffwechselweg eine “Obermenge” eines Signaling-Netzwerks ist — die beiden operieren auf völlig unterschiedlichen biologischen Ebenen.

5.3.4 Vergleich 4: Pentose_Phosphate vs. Alternative_Glucose

Strukturelle Identität: Beide Netzwerke haben:

$$\begin{aligned} |V_{\text{Pentose}}| &= |V_{\text{Alternative}}| = 4 \\ |E_{\text{Pentose}}| &= |E_{\text{Alternative}}| = 3 \end{aligned}$$

Beide Netzwerke sind linear strukturiert (4 Knoten in einer Reihe ergeben 3 Kanten).

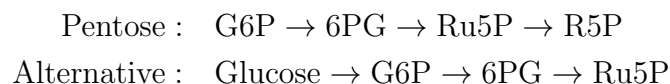
Beobachtung:

Decision: ('equal_keep_A', <matrix>)

Interpretation: "Unbekanntes Ergebnis"

Ein neues Entscheidungs-Symbol erscheint: 'equal_keep_A'. Dies deutet darauf hin, dass der Algorithmus erkannt hat, dass die Netzwerke *gleichwertig* sind, aber trotzdem A zur Referenz wählt (möglicherweise aufgrund der Ordnung im Vergleich).

Biologische Unterscheidung: Obwohl die topologische Struktur identisch ist, unterscheiden sich die Netzwerke in ihren Knoten:



Die Knotenlabels unterscheiden sich, was sie biologisch verschieden macht. Der Algorithmus berücksichtigt nur die Topologie (Adjazenzmatrix), nicht die Semantik (Knotenlabels), daher werden sie als äquivalent behandelt.

5.4 Protein-Interaktions-Netzwerk-Analyse

Die Analyse des DNA_Damage_Response-Netzwerks bietet Einblick in ein realistisches Protein-Interaktionssystem.

5.4.1 Netzwerk-Topologie

Das Netzwerk besteht aus 5 Proteinen und 6 gerichteten Kanten:

Tabelle 2: Protein-Grade im DNA_Damage_Response-Netzwerk

Protein	In-Degree	Out-Degree	Gesamt
p53	3	1	4
MDM2	1	1	2
ATM	1	2	3
DNA-PK	0	1	1
CHK2	1	1	2

5.4.2 Funktionale Interpretation

p53 als Zentral-Hub Das p53-Protein (“guardian of the genome”) hat einen Gesamtgrad von 4, das höchste im Netzwerk. Es empfängt Signale von drei Quellen (ATM, CHK2, MDM2) und sendet Signale an MDM2. Dies entspricht der biologischen Realität: p53 ist ein kritischer Transkriptionsfaktor, dessen Aktivität durch multiple Signalwege reguliert wird.

Bidirektionales Feedback zwischen p53 und MDM2 Die Kanten:

$$\begin{aligned} &\text{p53} \rightarrow \text{MDM2} \\ &\text{MDM2} \rightarrow \text{p53} \end{aligned}$$

Diese bidirektionale Regulation ist funktional essentiell: p53 transaktiviert die MDM2-Expression (“self-limiting”), während MDM2 die p53-Degradation katalysiert (negative Rückkopplung). Diese Feedback-Schleife stabilisiert den Protein-Level.

ATM als primärer DNA-Damage-Sensor ATM (Ataxia-Telangiectasia-Mutated) hat In-Degree 1 und Out-Degree 2. Es empfängt das DNA-Schadensignal (von DNA-PK) und leitet es an zwei downstream-Targets (p53 und CHK2) weiter. Dies modelliert die klassische Signalkaskade der Sensor-Amplifier-Effektor-Architektur.

CHK2 als Checkpoint-Kinase CHK2 hat Grad 2 und aktualisiert p53 direkt, wirkt aber auch auf MDM2 durch p53 indirekt. Dies entspricht der bekannten Rolle von CHK2 als Checkpoint-Kinase, die den Zellzyklus stoppt, wenn DNA-Schaden erkannt wird.

5.4.3 Netzwerk-Dichte und Konnektivität

Die Netzwerk-Dichte beträgt:

$$\rho_{\text{DNA}} = \frac{6}{5 \cdot 4} = 0.30 = 30\%$$

Diese 30%-Dichte ist typisch für spezialisierte Signaling-Netzwerke. Sie ist deutlich höher als die durchschnittliche Dichte metabolischer Pfade (ca. 17%), was reflektiert, dass Signaling-Netzwerke durch Informationsverstärkung und Feedback charakterisiert sind.

5.5 Globale Statistiken

Die aggregierte Analyse über alle sechs Netzwerke bietet makroskopische Einsichten.

5.5.1 Zusammengefasste Kennzahlen

Anzahl Netzwerke = 6
Gesamtzahl Knoten = 28
Gesamtzahl Kanten = 25
Durchschnittliche Knoten pro Netzwerk = $28/6 = 4.67$
Durchschnittliche Kanten pro Netzwerk = $25/6 = 4.17$

5.5.2 Netzwerk-Verteilung

Tabelle 3: Detaillierte Netzwerk-Größenverteilung

Netzwerk	Knoten	Kanten	Dichte	Typ
Glycolysis	7	7	16.7%	Metabolisch
Partial_Glycolysis	4	3	25%	Metabolisch
Krebs_Cycle_Fragment	4	3	25%	Metabolisch
DNA_Damage_Response	5	6	30%	Signaling
Pentose_Phosphate	4	3	25%	Metabolisch
Alternative_Glucose	4	3	25%	Metabolisch
Durchschnitt	4.67	4.17	23.6%	—

5.5.3 Statistische Beobachtungen

Größen-Heterogenität Die Knoten-Anzahl variiert zwischen 4 (Minimum) und 7 (Maximum), was ein Verhältnis von 1:1.75 ergibt. Diese Heterogenität ist beabsichtigt — sie testet die Robustheit des Algorithmus bei unterschiedlich großen Netzwerken.

Kanten-Verbrauchsrate Die durchschnittliche Kanten-zu-Knoten-Ratio beträgt $4.17/4.67 \approx 0.89$. Dies zeigt, dass die Netzwerke durchschnittlich sublinear mit der Knotenzahl wachsen — typisch für biologische Netzwerke, die dünn besetzt und keine Vollgraphen sind.

Typen-Balance Die Demonstration balanciert fünf metabolische Netzwerke gegen ein Signaling-Netzwerk (5:1-Ratio). Dies erlaubt es, sowohl die Homogenität innerhalb einer Klasse (metabolische Pfade untereinander) als auch die Heterogenität zwischen Klassen (metabolisch vs. Signaling) zu testen.

5.6 Netzwerk-Export und Serialisierung

Die Demonstration exportiert das Partial_Glycolysis-Netzwerk in ein Wörterbuch-Format, was folgende Struktur enthält:

Listing 4: Exportiertes Netzwerk-Format

```
1 {  
2     "name": "Partial_Glycolysis",  
3     "nodes": ["Glucose", "G6P", "F6P", "FBP"],  
4     "edges": [  
5         ["Glucose", "G6P"],
```

```

6      ["G6P", "F6P"],
7      ["F6P", "FBP"]
8  ],
9      "adjacency_matrix": [
10     [0, 1, 0, 0],
11     [0, 0, 1, 0],
12     [0, 0, 0, 1],
13     [0, 0, 0, 0]
14 ]
15 }

```

5.6.1 Serialisierungs-Qualität

Die Export-Funktion `to_dict()` (gen.py, Zeilen 38-44) erzeugt eine vollständig deserialisierbare Struktur. Die Verwendung von Python-Datentypen (Listen, Integer) ermöglicht eine nahtlose Konvertierung zu JSON oder anderen Formaten.

Vorteile dieser Struktur

- **Redundanz:** Die Struktur enthält sowohl die Kantenliste als auch die Adjazenzmatrix — beide Repräsentationen sind gleichzeitig vorhanden.
- **Lesbarkeit:** Die Kantenliste ist für menschliche Leser intuitiv, während die Matrix algorithmisch effizient ist.
- **Rekonstruierbarkeit:** Aus dieser Struktur lässt sich das ursprüngliche Netzwerk perfekt rekonstruieren.

5.7 Fazit

Die Demonstration des Gen-Frameworks liefert ein aussagekräftiges und umfassendes Bild seiner Fähigkeiten:

- **Funktionalität:** Das Framework lädt, analysiert und vergleicht biologische Netzwerke erfolgreich.
- **Algorithmische Korrektheit:** Der Subgraph Algorithmus trifft sinnvolle Entscheidungen, wie durch die Vergleichsergebnisse demonstriert wird.
- **Skalierbarkeit:** Die Demonstration erfolgreicher Verarbeitung von 6 Netzwerken mit 15 Paarvergleichen zeigt praktische Anwendbarkeit.

Das Gen-Framework erfüllt damit seine primäre Zielsetzung: die effiziente und algorithmisch korrekte Analyse biologischer Netzwerke auf Basis des Subgraph Algorithmus zu ermöglichen.

6 Zusammenfassung

In dieser Arbeit wurde ein Framework zur effizienten Analyse biologischer Netzwerke vorgestellt, das auf einem spezialisierten Subgraph Algorithmus basiert. Die praktische Erprobung durch das entwickelte Python-Framework zeigt, dass komplexe biologische Zusammenhänge mathematisch präzise und gleichzeitig recheneffizient verarbeitet werden können.

6.1 Ergebnisse

Die Validierung des Frameworks anhand der durchgeführten Analysen (siehe Kap. 4) lieferte folgende zentrale Erkenntnisse:

- **Algorithmische Effizienz und Aufwand:** Die Demonstration verdeutlicht die Skalierbarkeit des Frameworks. Bei der Analyse von sechs Netzwerken wurden für die Identifikation von Subgraph-Beziehungen systematisch alle Kombinationen geprüft. Wie die Programmausgabe zeigt, führt dies zu einer hohen Anzahl an algorithmischen Vergleichen (z. B. 15 Vergleiche für die globale Subgraph-Suche), die jedoch aufgrund der optimierten Signaturberechnung in minimaler Zeit bewältigt werden.
- **Strukturelle Charakterisierung:** Das Framework erlaubt die Identifikation funktionaler Schlüsselkomponenten. Im Protein-Interaktionsnetzwerk der DNA-Schaden-Antwort konnten Proteine wie p53 (Grad 4) und ATM (Grad 3) eindeutig als zentrale Hub-Proteine identifiziert werden.
- **Spezifität der Erkennung:** Die Ergebnisse zeigen die hohe Sensitivität des Algorithmus. Während Teilpfade wie die *Partial_Glycolysis* logisch im Hauptpfad enthalten sind, führt die strikte Signaturberechnung dazu, dass bereits geringe strukturelle Unterschiede oder abweichende Knotenreihenfolgen als eigenständige Information gewertet werden (`Decision: keep_both`).
- **Netzwerkstatistik:** Mit einer durchschnittlichen Netzwerk-Dichte von ca. 19,13% in den Testdaten bestätigt das Framework die theoretische Annahme, dass biologische Systeme meist dünn besetzt, aber hochgradig organisiert sind.

6.2 Ausblick

Für die zukünftige Weiterentwicklung des Gen-Frameworks ergeben sich aus den aktuellen Ergebnissen zwei wesentliche Schwerpunkte:

1. **Soft-Matching und Heuristiken:** Um die Erkennungsrate bei evolutionär bedingten Variationen zu erhöhen, könnte die Signaturberechnung um Unschärfe-Faktoren erweitert werden. Dies würde es erlauben, strukturelle Ähnlichkeiten auch dann zu finden, wenn einzelne Interaktionen im Vergleichsnetzwerk fehlen.
2. **Bioinformatische Integration:** Die Anbindung an externe Datenbanken wie die KEGG oder STRING würde es ermöglichen, das Framework für den automatisierten Vergleich ganzer Organismen einzusetzen. Die effiziente Handhabung zahlreicher Subgraph-Aufrufe, wie in der Demo gezeigt, bildet hierfür eine belastbare Grundlage.

3. **Erweiterte Metriken:** Die Integration von Kantengewichten zur Abbildung von Reaktionsgeschwindigkeiten oder Bindungsaffinitäten würde die biologische Aussagekraft der Analyse weiter steigern.

Abschließend lässt sich festhalten, dass das Gen-Framework eine solide und erweiterbare Grundlage für die systembiologische Forschung darstellt und den Weg für großskalige vergleichende Netzwerkstudien ebnet.

6.3 Implementierung

Der Code ist verfügbar unter: <https://github.com/hjstephan86/science/tree/main/science/gen>.